多层感知机（第二部分）

前向传播、反向传播和计算图：梯度是怎么算出来的?

- **前向传播**：根据变量之间的关系式，从输入一层层算到输出
- **计算图**：根据变量之间的关系式画出来的计算流程图
- **反向传播**：从输出（损失）出发，一层一层回到输入，用链式法则求各个参数的梯度

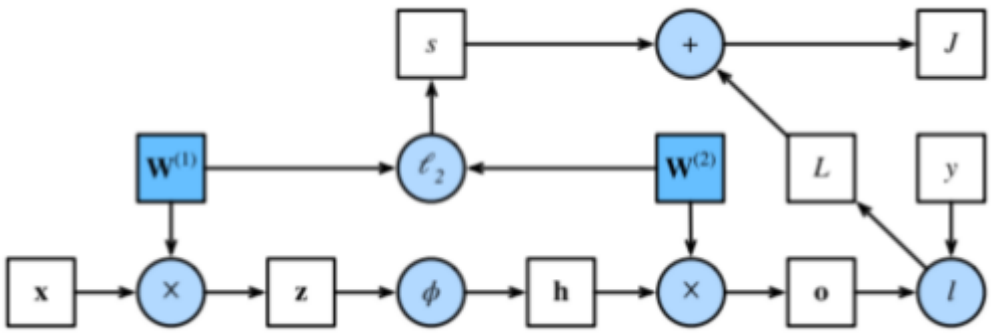
举例：输入为 $\mathbf{x}$，训练参数为 $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$，输出（损失）为 J

关系式：

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x}, \quad \mathbf{h} = \phi(\mathbf{z}), \quad \mathbf{o} = \mathbf{W}^{(2)}\mathbf{h}, \quad L = l(\mathbf{o}, y)$$

$$s = \frac{\lambda}{2}\left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2\right), \quad J = L + s$$

对应的计算图如下所示



反向传播：通过计算图，找到输出和输入之间的桥梁，通过链式法则求偏导数（梯度）。

完成以上内容 + 根据梯度更新参数即完成一轮训练。

由此可知：**训练比测试更耗内存**

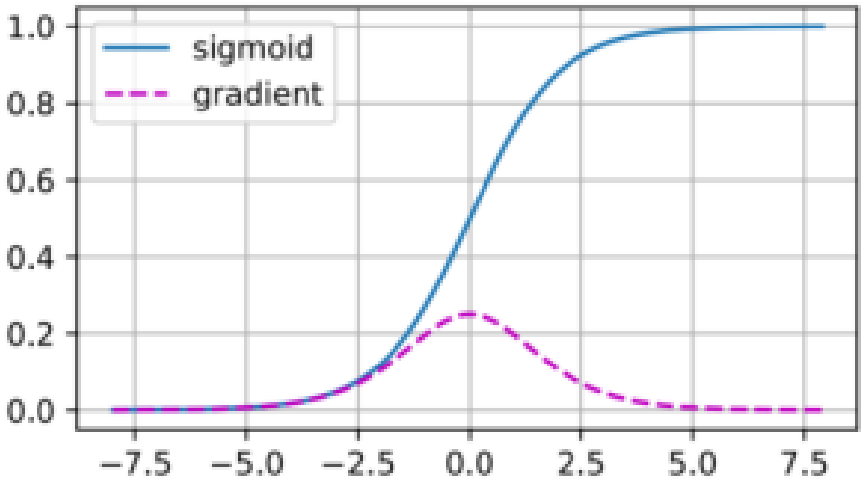
- 原因：测试时只需要前向传播，算出结果就行
- 训练时不仅要前向传播，还要为反向传播保留很多中间结果在内存中

数值稳定性和模型初始化

实际训练中会碰到三种问题：**梯度消失**、**梯度爆炸**、**对称性**

1. 梯度消失

以 sigmoid 函数为例，如下图所示，该函数在输入偏离 0 较远时梯度很小，趋近于 0



这就导致，在反向传播通过许多层时，总梯度等于各个环节梯度乘积：

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \prod_k \frac{\partial(\cdot)_{k+1}}{\partial(\cdot)_k}$$

若很多环节的梯度都趋向于 0，整个乘积的梯度可能会"消失"（太小以至于无法产生影响）。

所以一般会选择更稳定的 **ReLU 函数**（在输入大于 0 时梯度恒为 1）。

2. 梯度爆炸

同理，如果相乘的梯度都大于 1，结果数值会很大甚至溢出。

3. 对称性

对于一个隐藏层的多个单元，若初始化为同一个值，则在之后的训练中，这些单元始终保持同步变化，就像只有一个单元一样。这样浪费了算力。

为了解决以上问题，需要进行合适的**参数初始化**。

默认情况：用正态分布来初始化权重值

出现问题：如果某一层的输入很多，权重初始化太大 → 输出的方差会越来越大；权重初始化太小 → 输出和梯度越来越小

（换句话讲，每层神经元个数不同，如果还用同样大小的初始化，深层网络就容易不稳定）

解决方法：Xavier 初始化

根据这一层的输入维度和输出维度，**自动调节初始化尺度**，让信号和梯度在网络里传播时不会迅速放大或缩小。

具体做法（数学结论，不重要）：

如果某层输入单元数是 n_{in}，输出单元数是 n_{out}，那么 Xavier 初始化让权重的方差取成大约：

$$\sigma^2 = \frac{2}{n_{in} + n_{out}}$$

对应地，如果用均匀分布初始化，范围是：

$$U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

环境和分布偏移

前面探讨的都是理想情况下的训练（训练集和预测集服从同样的分布）。

但是，现实情况要复杂得多：例如环境随时间变化，训练时的数据集取自过去的时间，相较于测试集有时间上的落后，使得数据的规律等发生变化，可能导致模型失灵。

或者，模型的预测行为本身可能就会对环境造成影响和改变。

这种预测集和训练集的分布出现偏差的现象称之为**分布偏移**。

符号化语言

- $\mathbf{x}$：输入、特征、观测到的数据
- y：标签、要预测的目标
- $P(\mathbf{x}, y)$ 联合分布：表示输入和其对应标签一起出现的总体规律，表征环境
- $P(\mathbf{x})$ 输入的边沿分布：会出现什么样的输入，概率几何
- $P(y)$ 输出的边沿分布：会出现什么样的输出，概率几何
- $P(y \mid \mathbf{x})$：在给定输入的情况下，标签 y 的概率
- $P(\mathbf{x} \mid y)$：在给定输出标签的情况下，输入 $\mathbf{x}$ 的概率
- 下标 S：训练时；T：测试时

分布偏移主要有三类

1. 协变量偏移

看到的输入的样子变了，但是输入和输出的对应关系（规则）没变。

符号化：

$$p_S(\mathbf{x}) \neq p_T(\mathbf{x}), \quad p_S(y \mid \mathbf{x}) = p_T(y \mid \mathbf{x})$$

直观理解：考察同样的知识，训练时用选择题训练，测试时用填空题。

纠正方法：给样本**加权**，在测试分布中常见的**输入**样本的权重比罕见的权重大。

权重因子：

$$\beta_i = \frac{p(\mathbf{x}_i)}{q(\mathbf{x}_i)}$$

2. **标签偏移**

标签比例变了，但是标签对应的输入没有变。

符号化：

$$p_S(y) \neq p_T(y), \quad p_S(\mathbf{x} \mid y) = p_T(\mathbf{x} \mid y)$$

直观理解：疾病诊断中，以前某种病很少见，但是现在流行起来了。病人出现的比例变多，但是"得这种病的人通常出现的症状/表现"没变。

纠正方法：给样本**加权**，在测试分布中常见的**输出**样本的权重比罕见的权重大。

权重因子：

$$\beta_i = \frac{p(y_i)}{q(y_i)}$$

3. **概念偏移**

最彻底的变化，输入和输出的对应关系都变了（同一个输入，在训练和测试中对应了不同的输出）。

符号化：

$$p_S(y \mid \mathbf{x}) \neq p_T(y \mid \mathbf{x})$$

直观理解："时尚"的概念，随着时间变化，原先时尚的变得不时尚，不时尚的变得时尚。

纠正方法：由于变化前后毫无规律可言，一般来说没有很好的纠正方法。

- 若转变巨大、突然发生：抛弃现有模型，直接重新训练

- 若转变缓慢且幅度较小：可以用新数据持续更新现有模型，而不是完全从零开始

机器学习问题有很多不同形态，包括批量学习、在线学习、老虎机、控制、强化学习；它们的区别关键在于数据是否持续到来、环境是否会记住并响应你的行为，因此不能用同一种静态视角看所有学习问题。一旦模型进入真实系统，就不只是数学优化问题，还会牵涉公平、责任和社会影响。